
Projektdokumentation: Integration von MoveIt 2 Dual-Arm Teleoperation

Architektur-Erweiterung: Relative Steuerung (Unity $\rightarrow$ ROS 2 Humble $\rightarrow$ NVIDIA Isaac Sim)

Diese Dokumentation beschreibt die heute gelösten technischen Herausforderungen, die neue asynchrone Systemarchitektur und das implementierte relative Steuerungskonzept (Delta-Steuerung) zur simultanen Bewegung beider Arme des humanoiden Roboters Walker S2.

1. Gelöste Kernprobleme & Systemarchitektur

A. Behebung des Python-Versionskonflikts

- **Herausforderung:** NVIDIA Isaac Sim erfordert zwingend **Python 3.11** (`~/walker_ws_ros2`), während das native ROS 2 Humble auf Ubuntu 22.04 standardmäßig mit **Python 3.10** arbeitet. Der Versuch, MoveIt 2-Knoten direkt mit den für Python 3.11 kompilierten UBTECH-Nachrichten (`mc_task_msgs`) zu starten, führte zu kritischen `ModuleNotFoundError`- und `UnsupportedTypeSupport`-Abstürzen.
- **Lösung:** Da ROS 2 über das Netzwerkprotokoll (DDS) vollkommen entkoppelt kommuniziert, haben wir das Nachrichtenpaket `mc_task_msgs` direkt im sauberen Übersetzer-Workspace `~/unity_ws` lokal für **Python 3.10** neu kompiliert. Beide Umgebungen laufen nun fehlerfrei auf ihrer jeweils nativen Python-Version und tauschen Daten nahtlos über das Netzwerk aus.

B. Behebung des Deadlocks im ROS 2 Brücken-Knoten

- **Herausforderung:** Die simultane Abfrage der inversen Kinematik (IK) für den linken und rechten Arm über den MoveIt 2 Service `/compute_ik` blockierte sich bei der Verwendung von traditionellen, synchronen Aufrufen gegenseitig (`spin_until_future_complete`). Dies führte zu einem vollständigen Einfrieren des Terminals (Deadlock).
 - **Lösung:** Der Brücken-Knoten (`walker_moveit_bridge`) wurde vollständig auf eine **asynchrone, nicht-blockierende Architektur** umgestellt. Durch den Einsatz einer `ReentrantCallbackGroup` und asynchronen Callbacks (`add_done_callback`) verarbeitet der Knoten die Datenströme aus Unity für beide Arme parallel und in Echtzeit.
-

2. Das neue relative VR-Steuerungskonzept

(Delta-Teleoperation)

Bei der absoluten Positionsübertragung führten minimale Abweichungen oder unnatürliche Handgelenksdrehungen in Unity sofort zum MoveIt-Fehler Kod: -31 (NO_IK_SOLUTION), da die Zielpunkte außerhalb der physikalischen Reichweite des Roboters lagen.

Heute haben wir das System erfolgreich auf das **"Relative VR-Steuerungsmodell"** umgestellt, welches exakt der späteren Funktionsweise der **Meta Quest Goggles** entspricht:

1. **Kalibrierung beim Start:** Beim Klick auf *Play* in Unity wird die aktuelle Position der Würfel (später der VR-Controller) als virtueller Nullpunkt gespeichert.
2. **Delta-Berechnung:** Bewegt man die Hand in Unity, berechnet das C#-Skript nur die relative Veränderung (**die Delta-Distanz**) zum Startpunkt.
3. **Mapping in die Komfortzone:** Dieses Delta wird im Python-Knoten auf eine vordefinierte, absolut sichere Erreichbarkeitszone vor der Brust des Roboters (bezogen auf den Koordinatenursprung `base_link` im Becken des Roboters) aufaddiert.
4. **3D-Rotations-Tracking (Quaternionen):** Die vollständige Rotation der Würfel wird nun live an ROS 2 übertragen. Dadurch passt MoveIt 2 die Orientierung des Roboter-Endeffektors dynamisch an, was die Gelenke (Schulter, Ellbogen, Handgelenk) entlastet und den Bewegungsspielraum drastisch vergrößert.

Erfolgreich validierter Bewegungsraum in den Logdaten:

- Rechte Hand erfolgreich bewegt von \$Z: 0.19\$ bis \$Z: 0.90\$ (Vertikalbewegung von Bauch- bis Kinnhöhe in Isaac Sim).

3. Die exakte Startprozedur (4 Terminals)

Um das Gesamtsystem zu reproduzieren, müssen die folgenden vier Komponenten in separaten Terminals in exakt dieser Reihenfolge gestartet werden:

Terminal 1: NVIDIA Isaac Sim (Python 3.11)

Bash

```
cd ~/walker_test
# Ggf. Isaac Sim Umgebung aktivieren (z.B. conda activate env_isaacsim)
source ~/walker_ws_ros2/install/setup.bash
python3 play_ros2.py
```

Terminal 2: MoveIt 2 Hauptgehirn (System-Python 3.10)

Bash

```
cd ~/moveit2_ros
source install/setup.bash
ros2 launch walkerS2_dual_arm_moveit_config demo.launch.py
```

Terminal 3: Unity TCP-Server (System-Python 3.10)

Bash

```
cd ~/unity_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run ros_tcp_endpoint default_server_endpoint --ros-args -p ROS_IP:=0.0.0.0
```

Terminal 4: MoveIt Brücken-Übersetzer (System-Python 3.10)

Bash

```
cd ~/unity_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run walker_moveit_bridge bridge_node
```

4. Aktueller Quellcode im Übersetzer-Workspace

C# Unity-Skript (MoveItDualArmPublisher.cs)

Das Skript berechnet die relative Verschiebung in Unity und übersetzt das Koordinatensystem (Unity: Left-Handed $\rightarrow$ ROS 2: Right-Handed, $X = \text{Vorne}$, $Y = \text{Links}$, $Z = \text{Oben}$).

C#

```
using UnityEngine;
```

```

using Unity.Robotics.ROSTCPConnector;
using RosMessageTypes.Geometry;

public class MoveItDualArmPublisher : MonoBehaviour
{
    private ROSConnection ros;
    public string leftArmTopic = "/unity/left_target_pose";
    public string rightArmTopic = "/unity/right_target_pose";

    public GameObject leftTargetCube;
    public GameObject rightTargetCube;

    private Vector3 initialLeftCubePos;
    private Vector3 initialRightCubePos;

    // Start-Komfortzone im ROS-Bezugssystem (40cm vor dem Roboter, 20cm über Becken)
    private Vector3 safeLeftRobotPos = new Vector3(0.4f, 0.3f, 0.2f);
    private Vector3 safeRightRobotPos = new Vector3(0.4f, -0.3f, 0.2f);

    private Vector3 lastLeftPos = Vector3.zero;
    private Vector3 lastRightPos = Vector3.zero;
    private const float threshold = 0.001f;

    void Start()
    {
        ros = ROSConnection.GetOrCreateInstance();
        ros.RegisterPublisher<PoseMsg>(leftArmTopic);
        ros.RegisterPublisher<PoseMsg>(rightArmTopic);

        if (leftTargetCube != null) { initialLeftCubePos = leftTargetCube.transform.position;
lastLeftPos = leftTargetCube.transform.position; }
        if (rightTargetCube != null) { initialRightCubePos = rightTargetCube.transform.position;
lastRightPos = rightTargetCube.transform.position; }
    }

    void Update()
    {
        if (leftTargetCube != null && Vector3.Distance(leftTargetCube.transform.position,
lastLeftPos) > threshold)
        {
            PublishRelativePose(leftTargetCube, initialLeftCubePos, safeLeftRobotPos, leftArmTopic);
            lastLeftPos = leftTargetCube.transform.position;
        }
    }
}

```

```

        if (rightTargetCube != null && Vector3.Distance(rightTargetCube.transform.position,
lastRightPos) > threshold)
        {
            PublishRelativePose(rightTargetCube, initialRightCubePos, safeRightRobotPos,
rightArmTopic);
            lastRightPos = rightTargetCube.transform.position;
        }
    }

    void PublishRelativePose(GameObject cube, Vector3 initialCubePos, Vector3 safeRobotPos,
string topic)
    {
        Vector3 deltaUnity = cube.transform.position - initialCubePos;
        Vector3 deltaROS = new Vector3(deltaUnity.z, -deltaUnity.x, deltaUnity.y);
        Vector3 finalTargetPos = safeRobotPos + deltaROS;

        PoseMsg msg = new PoseMsg
        {
            position = new PointMsg { x = finalTargetPos.x, y = finalTargetPos.y, z = finalTargetPos.z },
            orientation = new QuaternionMsg
            {
                x = cube.transform.rotation.z,
                y = -cube.transform.rotation.x,
                z = cube.transform.rotation.y,
                w = cube.transform.rotation.w
            }
        };
        ros.Publish(topic, msg);
    }
}

```